Scrum Simulation Project: Second Report (Group 7)

Smart Grocery Shopping List App

Group Members:

Keerthi Jangidi (Role: Product Owner)
Shreya Chidananda (Role: Scrum Master)
Srikala Sandu (Role:Development Team)
Preeti Sanjay Chougule (Role: Development Team)
Anjali Nalli (Role: Development Team)
Shweta Pattanaik (Role: Development Team)

Introduction

In the ever evolving digital world, traditional methods of grocery shopping are often flawed in terms of organization and efficiency. Forgotten items, excessive purchases, and food waste are common challenges faced by consumers. To address these issues, our smart grocery shopping list app has been designed to streamline and enhance the grocery shopping experience.

Our mobile application will help users create and manage their shopping lists easily. Key features include adding, editing, and categorising grocery items, while also offering smart recommendations based on previous purchases. Additionally, expiration alerts for perishables and meal planning suggestions simplify shopping decisions.

To improve efficiency, the app integrates barcode scanning for seamless pantry tracking, and it also supports real time shared lists, enabling better collaboration among friends and family. By combining ai powered recommendations, location based reminders, and collaborative features, this app transforms the traditional grocery list into an intelligent shopping assistant that promotes smarter shopping habits, minimises waste, and saves valuable time.

Overview of Scrum Process:

Objective:
Enhance the app's core functionality by introducing innovative features and improving collaborative grocery list management.

Key Sprint 2 Deliverables:

- **Innovative Suggestions & Reminders**: Suggested items based on frequent purchases.

- **Pantry Inventory Tracking**: Items can be marked as in-pantry to prevent overbuying.
- **Meal Planning Assistance**: Suggested meals based on pantry contents.
- **Real-time Collaboration**: Shared lists for households with live sync.
- **Barcode Scanner Integration**: Improved recognition and manual entry for unknown items.
- **Expiration Alerts**: Notifications for perishables close to expiry.

Improvements Over Sprint 1:

- More focused planning with better task breakdown.
- UI/UX improvements from user testing.
- Stakeholder feedback was actively incorporated.

Tools Used:

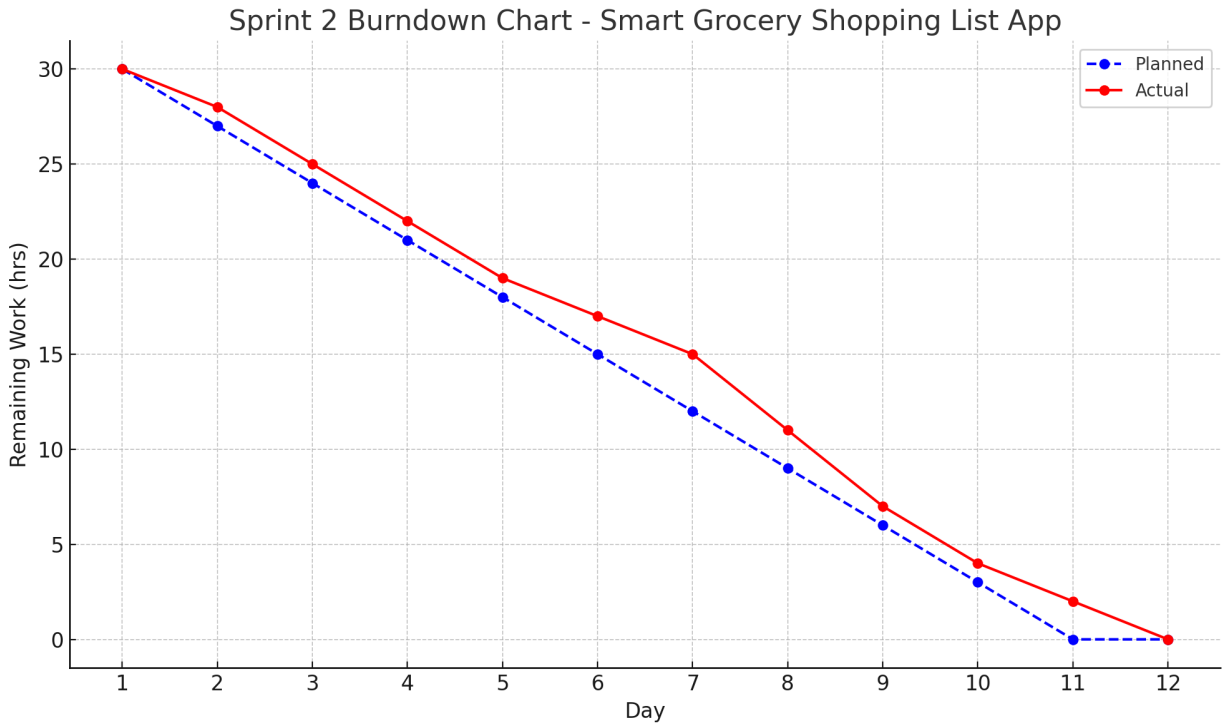
- React for the frontend
- Open Food Facts API for barcode scanning

Basic state management via React hooks

Burndown Chart:

Day	Planned Remaining Work (hrs)	Actual Remaining Work (hrs)
Day 1	30	30
Day 2	27	28
Day 3	24	25
Day 4	21	22
Day 5	18	19
Day 6	15	17
Day 7	12	15
Day 8	9	11

Day 9	6	7
Day 10	3	4
Day 11	0	2
Day 12	0	0



Product Backlog:

The Smart Grocery Shopping List App is designed to simplify and enhance the grocery shopping experience through intelligent features and seamless usability

The summary of the User Stories is as follows:

1. **Basic Grocery List Management**
   Enables users to create, edit, delete, and categorise grocery items, helping them stay

organised and manage their shopping effectively.

2. **Smart Suggestions & Reminders**
   Provides personalised item suggestions based on shopping history, sends reminders for perishable items nearing expiry, and issues location-based notifications when near a grocery store to prevent forgotten purchases.

3. **Pantry Inventory Tracking**
   Integrates barcode scanning for automatic inventory updates, displays current pantry contents, and allows users to assign expiration dates to items to minimise waste and unnecessary purchases.

4. **Meal Planning and Recipe Integration**
   It recommends meal plans and recipes based on the grocery list and pantry inventory while enabling users to seamlessly add missing recipe ingredients to their list.

5. **Multi-User Collaboration**
   Facilitates grocery list sharing and real-time updates among family members or housemates, ensuring synchronised shopping and reducing duplicate purchases.

Sprint Backlog:

- **For Sprint 1, the following tasks were carried out (with a time estimate of 1 week):**
  - Create, edit, and delete items
  - Categories of grocery items
  - Simple UI/UX design
  - Basic data storage

- **For Sprint 2, the following additional tasks were carried out (with a time estimate of 2 weeks):**
  - Smart reminders for frequently bought items
  - Expiry tracking for perishable goods
  - Barcode scanning for inventory tracking
  - Recipe recommendations based on pantry items
  - Shared grocery lists for family members
  - Real-time updates when someone buys an item

Sprint Review

After Sprint 2, the development team conducted a Sprint Review meeting to present the completed product increment to stakeholders and gather constructive feedback. This review aimed to demonstrate the functional enhancements made during the sprint, evaluate progress against the Sprint Goal, and collaboratively assess the next steps in product development.

Overview of Completed Work

The following features were successfully developed, tested, and integrated into the Smart Grocery Shopping List App:

1. Real-Time Shared Lists
   A collaborative grocery list feature was implemented, allowing multiple users to access and modify the same list simultaneously. Updates made by one user were reflected in real time across devices, significantly enhancing the app's utility for families and shared households.
2. Enhanced Barcode Scanning and Pantry Integration
   Integration with the Open Food Facts database improved the barcode scanner, resulting in better recognition of grocery items. Users were given the option to enter item details manually for unrecognised barcodes. Additionally, scanned items were automatically added to the pantry inventory, establishing the foundation for expiration tracking and pantry management.
3. Pantry Inventory and Expiration Alerts
   Users could now view a categorised pantry inventory and receive notifications for items approaching their expiration date. This feature was designed to reduce food waste and assist users with informed shopping and meal planning.
4. User Interface and Experience Enhancements
   Several refinements were made to the app's UI/UX, including improved item categorisation, simplified editing, and better visibility of list actions. These improvements were based on user feedback from Sprint 1 and contributed to an overall more intuitive user experience.

Sprint Review Outcome:

- Basic grocery list functionality works.
- Users can add, edit, and categorise items.
- Users receive innovative suggestions.
- Inventory auto-updates based on purchases
- User testing feedback incorporated for minor UI changes.
- Feedback suggests adding meal planning in the next sprint.

Stakeholder Feedback

Stakeholders responded positively to the enhancements demonstrated during the review. The real-time collaboration feature was particularly well received, and the barcode scanning improvements were acknowledged as a meaningful upgrade to the app's core functionality. However, stakeholders also noted the continued presence of duplicate items in grocery lists and recommended implementing an innovative duplicate detection system. This feedback has been reported and will be addressed in the subsequent sprint through natural language processing (NLP)-based logic.

Additional feedback included:

- Improving the speed and responsiveness of the barcode scanner.
- Enhancing offline functionality for real-time list updates.
- Introducing personalised shopping suggestions based on user habits.

Outcome and Evaluation

The Sprint Goal of enhancing collaboration, inventory tracking, and grocery list management was successfully achieved. All user stories committed in the Sprint Backlog were completed and integrated into the working product increment. The review validated the team's focus on delivering user centred features and demonstrated tangible progress in transforming the app into an intelligent and collaborative shopping assistant.

The feedback gathered during the review will inform the planning and prioritisation, with particular attention to implementing NLP driven duplicate detection, improving app performance, and adding personalised recommendation capabilities.

Sprint Retrospective

What Went Well

- **Clear Sprint Goal:** The team had a well-defined, realistic sprint goal that effectively guided development.
- **Collaborative Standups:** Daily standups helped identify blockers early and maintain team alignment.
- **Core Features Delivered:** The essential functionalities of adding, editing, deleting, and categorising grocery items were successfully implemented.
- **Stakeholder Engagement:** Valuable and actionable feedback was received during the Sprint Review, boosting confidence and providing strategic direction.

What Could Have Been Improved

- **Mid-Sprint Delays:** Some tasks experienced delays due to unforeseen technical issues, as reflected in the burndown chart.
- **Shared List Confusion:** Lack of real-time notifications led to minor miscommunications in shared grocery lists.
- **Manual Data Loss:** No recovery mechanism existed for accidentally deleted items or lists.

What We'll Do Differently in the Next Sprint

- **Implement a Smart Duplicate Detector:** Use NLP techniques to identify and merge similar items before adding them to the list.
- **Enhance Shared List Functionality:** Add real-time syncing and notification features to alert users of changes and prevent redundant purchases.
- **Optimise Sprint Planning:** Allocate buffer time for debugging and unexpected technical challenges to reduce mid-sprint delays.

Final Product

Code:

```
import React, { useState, useEffect } from 'react';


const App = () => {

  const [item, setItem] = useState('');

  const [category, setCategory] = useState('General');

  const [items, setItems] = useState([]);

  const [editingId, setEditingId] = useState(null);

  const [suggestions] = useState(['Milk', 'Eggs', 'Bread', 'Tomatoes', 'Bananas']);

  const [mealSuggestions, setMealSuggestions] = useState([]);
```

```
const mealsDB = [
  { name: "Omelette", ingredients: ["Eggs", "Milk"] },
  { name: "BLT Sandwich", ingredients: ["Bread", "Tomatoes"] },
  { name: "Banana Smoothie", ingredients: ["Bananas", "Milk"] },
  { name: "Salad", ingredients: ["Tomatoes"] },
];

const addItem = () => {
  if (!item.trim()) return;

  if (editingId !== null) {
    setItems(items.map(i =>
      i.id === editingId ? { ...i, name: item.trim(), category } : i
    ));
    setEditingId(null);
  } else {
    setItems([...items, {
      name: item.trim(),
      category,
      id: Date.now(),
      inPantry: false
    }]);
```

```
  }

  setItem('');
  setCategory('General');
};

const deleteItem = (id) => {
  setItems(items.filter(i => i.id !== id));
  if (editingId === id) {
    setItem('');
    setCategory('General');
    setEditingId(null);
  }
};

const editItem = (id) => {
  const toEdit = items.find(i => i.id === id);
  if (toEdit) {
    setItem(toEdit.name);
    setCategory(toEdit.category);
    setEditingId(id);
  }
};
```

```
const togglePantry = (id) => {
  setItems(prevItems =>
    prevItems.map(i =>
      i.id === id ? { ...i, inPantry: !i.inPantry } : i
    )
  );
};

useEffect(() => {
  const hasMilk = items.some(i => i.name.toLowerCase() === 'milk');
  if (hasMilk) {
    const timer = setTimeout(() => {
      alert("⏰ Don't forget to restock Milk!");
    }, 10000);
    return () => clearTimeout(timer);
  }
}, [items]);

useEffect(() => {
  const pantryList = items
    .filter(i => i.inPantry)
    .map(i => i.name.trim().toLowerCase());
```

```
  const suggestions = mealsDB.map(meal => {
    const missing = meal.ingredients.filter(
      ing => !pantryList.includes(ing.trim().toLowerCase())
    );
    return {
      name: meal.name,
      ingredients: meal.ingredients,
      missing
    };
  });

  setMealSuggestions(suggestions);
}, [items]);

return (
  <div style={{ padding: '20px', maxWidth: '700px', margin: '0 auto' }}>
    <h1 style={{ fontSize: '24px', fontWeight: 'bold', marginBottom: '20px' }}>Smart Grocery List</h1>

    <div style={{ marginBottom: '20px', padding: '20px', border: '1px solid #ccc', borderRadius: '8px' }}>
      <input
        type="text"
```

```
        placeholder="Enter item..."
        value={item}
        onChange={(e) => setItem(e.target.value)}
        style={{ padding: '10px', width: '100%', marginBottom: '10px' }}
      />

      {item && (
        <div style={{ marginBottom: '10px' }}>
          {suggestions
            .filter(s => s.toLowerCase().includes(item.toLowerCase()) && s.toLowerCase() !== item.toLowerCase())
            .map((s, idx) => (
              <div
                key={idx}
                onClick={() => setItem(s)}
                style={{
                  cursor: 'pointer',
                  padding: '5px 10px',
                  backgroundColor: '#eee',
                  borderRadius: '4px',
                  marginBottom: '4px'
                }}
              >
                {s}
```

```
          </div>
        ))}
      </div>
    )}

    <select
      value={category}
      onChange={(e) => setCategory(e.target.value)}
      style={{ padding: '10px', width: '100%', marginBottom: '10px' }}
    >
      <option value="General">General</option>
      <option value="Fruits">Fruits</option>
      <option value="Vegetables">Vegetables</option>
      <option value="Dairy">Dairy</option>
      <option value="Bakery">Bakery</option>
    </select>

    <button onClick={addItem} style={{ padding: '10px', width: '100%' }}>
      {editingId ? 'Update Item' : 'Add Item'}
    </button>
  </div>

  <div style={{ display: 'flex', flexDirection: 'column', gap: '10px' }}>
```

```
{items.map(({ id, name, category, inPantry }) => (
  <div key={id} style={{ display: 'flex', justifyContent: 'space-between', padding: '10px', border: '1px solid #ccc', borderRadius: '6px' }}>
    <div>
      <p style={{ fontWeight: '600', margin: 0 }}>{name}</p>
      <p style={{ fontSize: '14px', color: '#666', margin: 0 }}>{category}</p>
      <p style={{ fontSize: '12px', color: inPantry ? 'green' : '#999', marginTop: '4px' }}>
        {inPantry ? 'In Pantry' : 'Not in Pantry'}
      </p>
    </div>
    <div style={{ display: 'flex', gap: '6px' }}>
      <button onClick={() => togglePantry(id)} style={{ color: 'white', backgroundColor: 'green', border: 'none', padding: '6px 10px', borderRadius: '4px' }}>
        {inPantry ? 'Remove' : 'Add to Pantry'}
      </button>
      <button onClick={() => editItem(id)} style={{ color: 'white', backgroundColor: 'blue', border: 'none', padding: '6px 10px', borderRadius: '4px' }}>
        Edit
      </button>
      <button onClick={() => deleteItem(id)} style={{ color: 'white', backgroundColor: 'red', border: 'none', padding: '6px 10px', borderRadius: '4px' }}>
        Delete
      </button>
    </div>
  </div>
```

```
  ))}
</div>

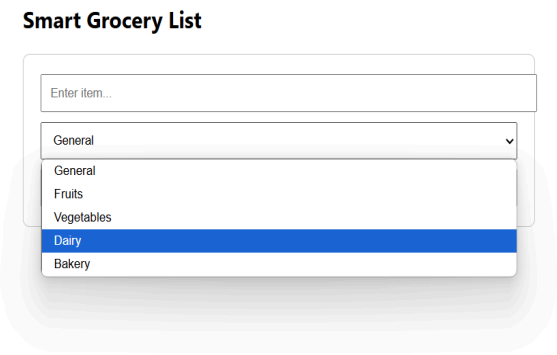
<div style={{ marginTop: '30px' }}>
  <h2 style={{ fontSize: '18px', marginBottom: '10px' }}>🧺 Pantry Inventory</h2>
  {items.filter(i => i.inPantry).length === 0 ? (
    <p style={{ fontSize: '14px', color: '#666' }}>No items in pantry yet.</p>
  ) : (
    <ul style={{ paddingLeft: '20px' }}>
      {items
        .filter(i => i.inPantry)
        .map((p, idx) => (
          <li key={idx}>{p.name}</li>
        ))}
    </ul>
  )}
</div>

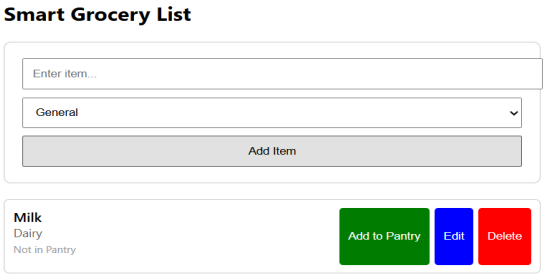
<div style={{ marginTop: '30px' }}>
  <h2 style={{ fontSize: '18px', marginBottom: '10px' }}>🍽️ Meal Suggestions</h2>
  {mealSuggestions.length === 0 ? (
    <p style={{ fontSize: '14px', color: '#666' }}>No meal data available.</p>
  ) : (
```

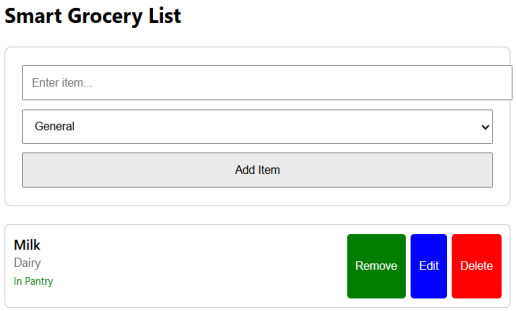
```
        <ul style={{ paddingLeft: '20px' }}>
          {mealSuggestions.map((meal, idx) => (
            <li key={idx} style={{ marginBottom: '10px' }}>
              <strong>{meal.name}</strong>
              {meal.missing.length === 0 ? (
                <span style={{ color: 'green', marginLeft: '10px' }}>✅ Ready to make!</span>
              ) : (
                <span style={{ color: 'orange', marginLeft: '10px' }}>
                  🛒 Missing: {meal.missing.join(', ')}
                </span>
              )}
            </li>
          ))}
        </ul>
      )}
    </div>
  </div>
 );
};

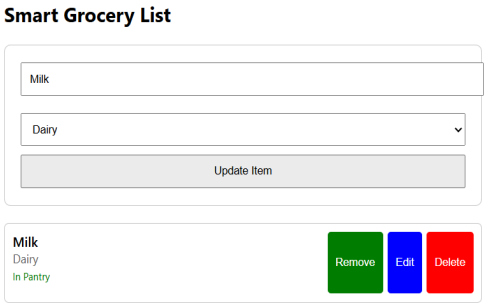
export default App;
```

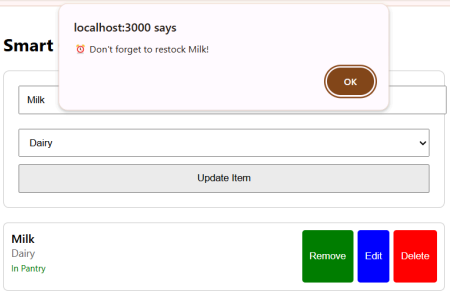
Screenshots of the Application

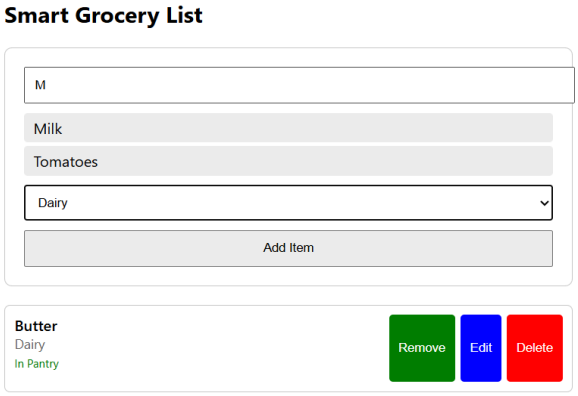


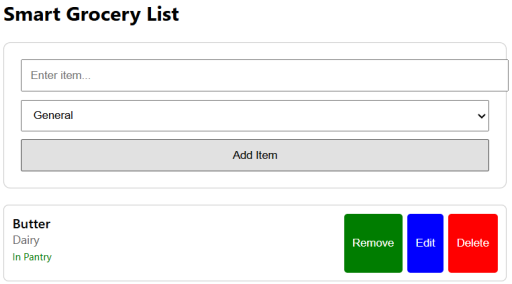


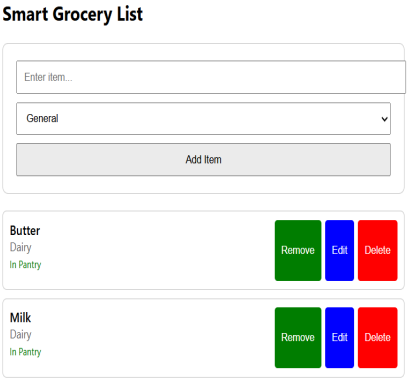


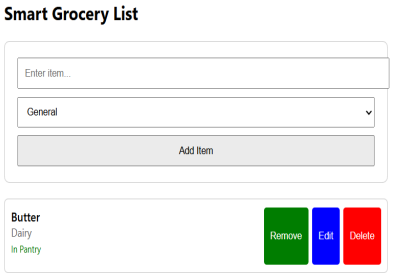

Smart Grocery List
Milk
Dairy
Update Item
Milk
Dairy
In Pantry
Remove
Edit
Delete


localhost:3000 says
Don't forget to restock Milk!
OK
Smart
Milk
Dairy
Update Item
Milk
Dairy
In Pantry
Remove
Edit
Delete


Smart Grocery List
M
Milk
Tomatoes
Dairy
Add Item
Butter
Dairy
In Pantry
Remove
Edit
Delete


Smart Grocery List
Enter item...
General
Add Item
Butter
Dairy
In Pantry
Remove
Edit
Delete


Smart Grocery List
Enter item...
General
Add Item
Butter
Dairy
In Pantry
Remove
Edit
Delete
Milk
Dairy
In Pantry
Remove
Edit
Delete


Smart Grocery List
Enter item...
General
Add Item
Butter
Dairy
In Pantry
Remove
Edit
Delete

Conclusion

Sprint 2 marked a significant leap forward in transforming our grocery list app into a smart, collaborative shopping assistant. The team successfully implemented core features such as real-time shared lists, pantry inventory tracking, and personalized reminders. User feedback from Sprint 1 was effectively incorporated and enhanced user satisfaction.

Despite minor mid-sprint delays and challenges with shared list notifications, all user stories from the Sprint Backlog were completed. Stakeholder engagement remained strong, and their valuable suggestions such as NLP-driven duplicate detection and improved offline support have been noted for future implementation.

Overall, Sprint 2 not only met its goal but also laid a solid foundation for intelligent features in the upcoming sprint, aligning the product closely with user needs and real-world grocery shopping behaviors.